

tempssm Reference Manual

Version 0.3.0

Akira S. Hirao, Shinya Baba, and Momoko Ichinokawa

2026-07-29

Summary

tempssm is an R package for state-space modeling of environmental temperature time series, including air and water temperature observations. It provides a practical framework for assessing how long-term trend, seasonal variation, autoregressive dependence, and optional exogenous effects contribute to observed temporal variation. The package facilitates the application of linear Gaussian state-space models estimated by Kalman filtering and smoothing, using the **KFAS** package as the computational backend (Helske, 2017).

Key Features

- Fits linear Gaussian state-space models to environmental temperature time series.
- Represents temperature dynamics using interpretable latent components: long-term trend, seasonal variation, autoregressive dependence, and optional exogenous effects.
- Supports arbitrary seasonal frequencies, while the current examples and validation focus primarily on monthly temperature data.
- Allows users to specify an arbitrary order of the autoregressive component (default: `AR(1)`).
- Includes time-series cross-validation tools for model evaluation.

Input Data Format

The core modeling functions in **tempssm** expect input time series to be supplied as R **ts** objects. The **ts** class is base R's standard format for regularly spaced time series (see the `stats::ts` documentation: <https://search.r-project.org/R/refmans/stats/html/ts.html>). The package also provides utility functions for converting common tabular and observational data formats into **ts** objects before model fitting.

The seasonal cycle used by `tempssm()` is taken from the `frequency` attribute of the input **ts** object. For example, `frequency = 12` represents monthly data.

Prior Art and Scope

tempssm provides a domain-focused workflow for analyzing temperature time series with linear Gaussian state-space models. It brings together model construction, component extraction, uncertainty summaries, residual diagnostics, visualization, and time-series cross-validation in a single R package interface tailored to temperature applications.

The package builds on established statistical methodology, including linear Gaussian state-space modeling, Kalman filtering, and Kalman smoothing. Model estimation is handled through the **KFAS** package, which provides a general framework for state-space models in R.

The initial implementation was adapted from the supplementary code provided by Baba (2024), accompanying Baba et al. (2024), which analyzed sea temperature trends using a linear Gaussian state-space model. The supplementary code is publicly available at:

<https://github.com/logics-of-blue/sea-temperature-trend-jogashima>

Compared with that prior implementation, `tempssm` extends the workflow into a reusable R package interface with input validation, documented S3 methods, tests, diagnostics, cross-validation utilities, and examples for broader temperature time-series analysis.

Model Overview

The model implemented in `tempssm` can be viewed as an extension of the Basic Structural Time Series Model (BSTSM), a standard state-space formulation that represents an observed time series using latent trend, seasonal, and irregular components. Following the temperature time-series application of Baba et al. (2024), `tempssm` keeps this interpretable decomposition and adds autoregressive dependence and optional exogenous effects. This makes it useful for separating long-term temperature change, seasonal cycles, and short-term departures.

The computational backend is KFAS, and the fitted `tempssm` object stores both filtering and smoothing estimates. Unless otherwise stated, this manual interprets summary statistics, diagnostics, and plots based on smoothed state estimates.

The detailed mathematical formulation is separated from this tutorial-style manual to reduce duplication and improve maintainability. See the package model specification document for the observation equation, state equations, seasonal constraint, autoregressive and exogenous components, parameter count, and estimation procedure. It is available as a PDF in the package repository:

<https://github.com/akihirao/tempssm/blob/main/vignettes/model-specification.pdf>

How to Use

Set Environment

Load the following R packages to run the examples below. If any packages are not installed, please install them as needed.

```
## Set libraries
library(tempssm)
library(forecast)

library(purrr)
library(tibble)
library(readr)
library(dplyr)
library(ggplot2)

library(patchwork)
```

Exercise I: Applying a State-Space Model to a Univariate Temperature Time Series

Objective

In Exercise I, we apply a linear Gaussian state-space model to a univariate temperature time series. This exercise introduces the basic workflow for model fitting, component visualization, residual diagnostics, and

short-term prediction without exogenous variables.

Loading the Sea Surface Temperature (SST) Dataset

A sample sea surface temperature (SST) dataset is included in the package.

- **Dataset:** Monthly sea surface temperature (SST) off Jogashima, Japan
- **Unit:** °C
- **Period:** January 1998 to February 2023

This dataset is a simulated monthly SST time series generated from the state-space model analysis of sea temperature off Jogashima reported by Baba et al. (2024). It is included as a reproducible example connected to the motivating study of this package.

```
data(sst_sim) # load a ts object of SST off Jogashima
head(sst_sim)
```

```
##           Jan   Feb   Mar   Apr   May   Jun
## 1998 16.19 13.82 16.44 16.49 19.70 21.83
```

```
summary(sst_sim)
```

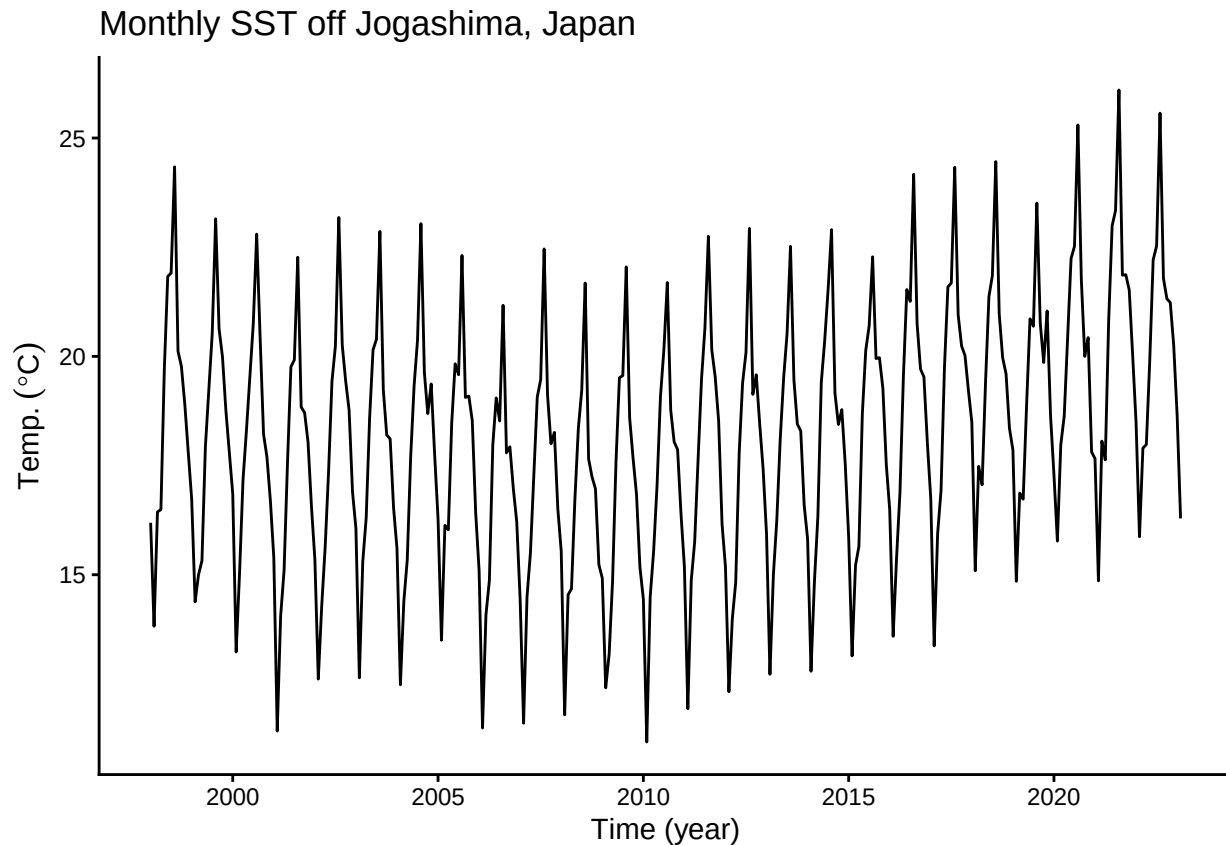
```
##           Temp
## Min.      :11.17
## 1st Qu.:16.22
## Median :18.39
## Mean    :18.23
## 3rd Qu.:20.11
## Max.    :26.10
```

Plotting the Monthly SST Time Series

We begin by visualizing the monthly SST time series to examine its overall structure, including apparent trends, seasonal variability, and the presence or absence of missing observations.

```
plt_sst_sim <- forecast::autoplot(sst_sim) +
  labs(y = expression(Temp.~(degree*C)),
       x = "Time (year)") +
  ggtitle("Monthly SST off Jogashima, Japan") +
  theme_classic()

plot(plt_sst_sim)
```



The overall mean SST is approximately 18.2 °C, and a clear seasonal pattern is visible. The series contains 0 missing observations. The raw time series suggests that SST may have decreased gradually from the beginning of the record to around 2008 and then increased thereafter. However, interannual variability is also evident, making the long-term pattern difficult to assess from the raw series alone.

Applying a Linear Gaussian State-Space Model

When a `ts` object containing temperature time-series data (here, `sst_sim`) is passed to the core function `tempssm()`, model construction and parameter estimation are performed together. The returned S3 object of class `tempssm` (here, `res_ar1`) stores the filtering and smoothing estimates, as well as the constructed model and input data. By default, `tempssm()` fits a first-order autoregressive model.

```
# model with first-order autoregressive component
res_ar1 <- tempssm(sst_sim) # (ar_order=1: default)
summary(res_ar1)
```

```
## tempssm summary
## -----
## Call:
## tempssm(temp_data = sst_sim)
##
## Model fit:
## Likelihood type: marginal
## Log-likelihood : -198.19
## k : 5
## Diffuse states : 13
## Converged : TRUE
##
```

```
## Variance parameters:
##   Observation (H): 0.07496416
##   State (Q trend): 4.937122e-06
##   State (Q season): 0.0001763538
##   State (Q ar): 0.1202156
##
## Components of auto-regression:
##   Order of AR: 1
##   Coefficient of AR1: 0.7579372
```

From the summary output, confirm that the model has converged (**Converged: TRUE**). The output also reports statistics such as the number of parameters (**k**), the log-likelihood, the likelihood type, and the number of diffuse initial states. The parameter estimates include the observation error variance (**H**), the process error variance of the long-term trend component (**Q trend**), the process error variance of the seasonal component (**Q season**), the process error variance of the autoregressive component, and the first-order autoregressive coefficient (**AR1**).

The log-likelihood and the associated number of estimated parameters can also be extracted directly from the fitted `tempssm` object using `logLik()`.

```
ll <- logLik(res_ar1)
ll
```

```
## 'log Lik.' -198.1929 (df=5)
```

```
attr(,"df") # number of parameters
```

```
## [1] 5
```

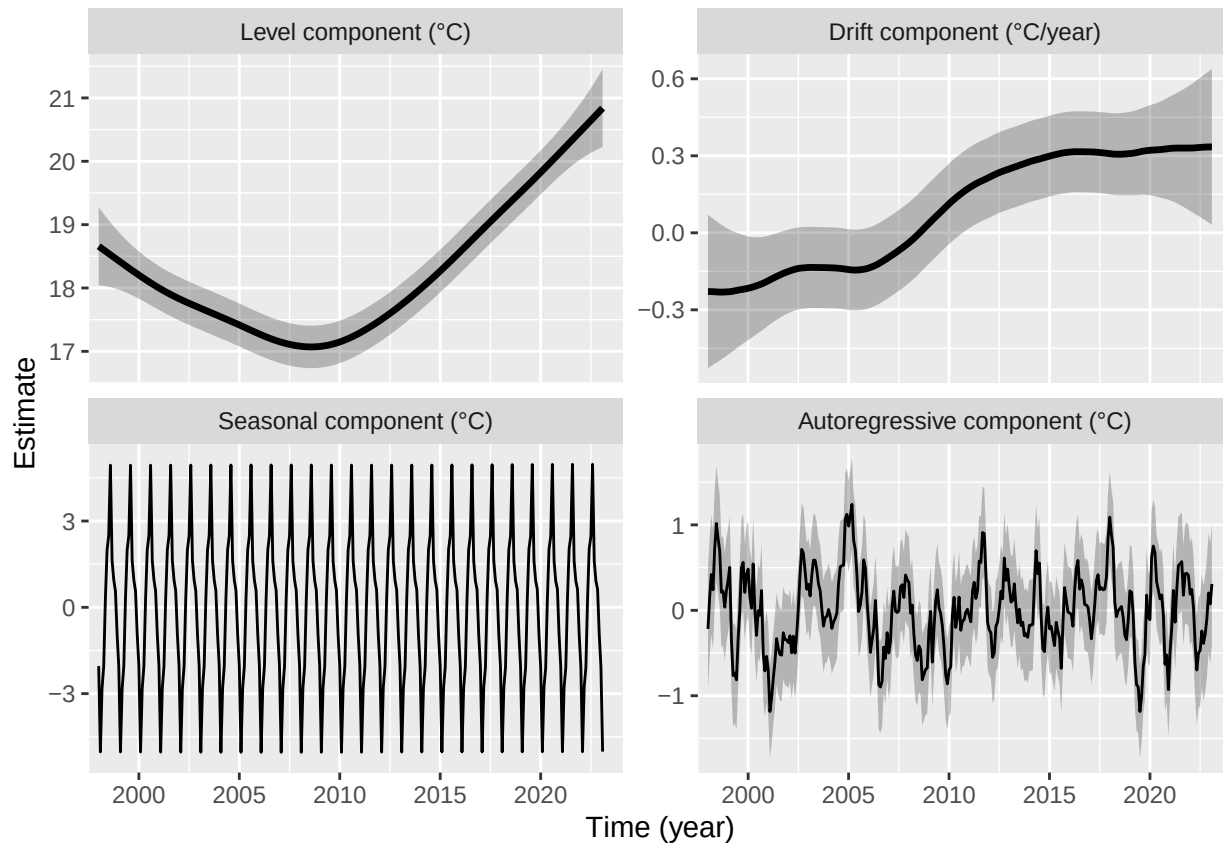
By default, `tempssm()` uses the KFAS marginal likelihood for parameter estimation. The diffuse likelihood remains available by explicitly setting `marginal = FALSE`. The selected likelihood type is stored in the fitted `tempssm` object and is used consistently by downstream methods such as `logLik()` and `summary()`.

The package intentionally does not compute AIC for `tempssm` objects. This is because AIC comparisons for state-space models can require care when likelihood type, diffuse initialization, or latent-state structure differs across models. To avoid presenting AIC as a default model-selection criterion, `tempssm` does not provide automatic AIC-based comparison. Instead, this manual emphasizes a model-assessment workflow based on residual diagnostics and time-series cross-validation. The log-likelihood and parameter count remain available through `logLik()` for users who need them for their own model-assessment workflows.

Plotting Long-Term Trend, Drift, Seasonal, and Autoregressive Components

We extract the corresponding latent components from the state-space model and visualize the estimated long-term evolution of temperature levels and their rate of change (drift). Plotting seasonal variation and autoregressive dependence as well makes the underlying trend structure easier to examine.

```
# plot all components at once
plot(res_ar1)
```



The long-term trend in the upper-left panel suggests that SST decreased during the first part of the study period and then increased after around 2008. In this example, the average annual rate of SST change over the full period is approximately 0.088 °C.

However, this full-period average should be interpreted together with the time-varying drift component. In the upper-right panel, the drift component is negative before around 2008 and becomes positive thereafter, corresponding to the shift from a decreasing to an increasing long-term SST pattern. The shaded gray areas represent 95% confidence intervals for the estimated latent states and illustrate the uncertainty associated with each estimated component.

For scripted workflows, `plot_tempssm_components()` returns the same component plot as a `ggplot` object; `ggplot2::autoplot()` is also available. To obtain a `ggplot` object for an individual component, specify the `component` argument in `plot_tempssm_components()` as follows.

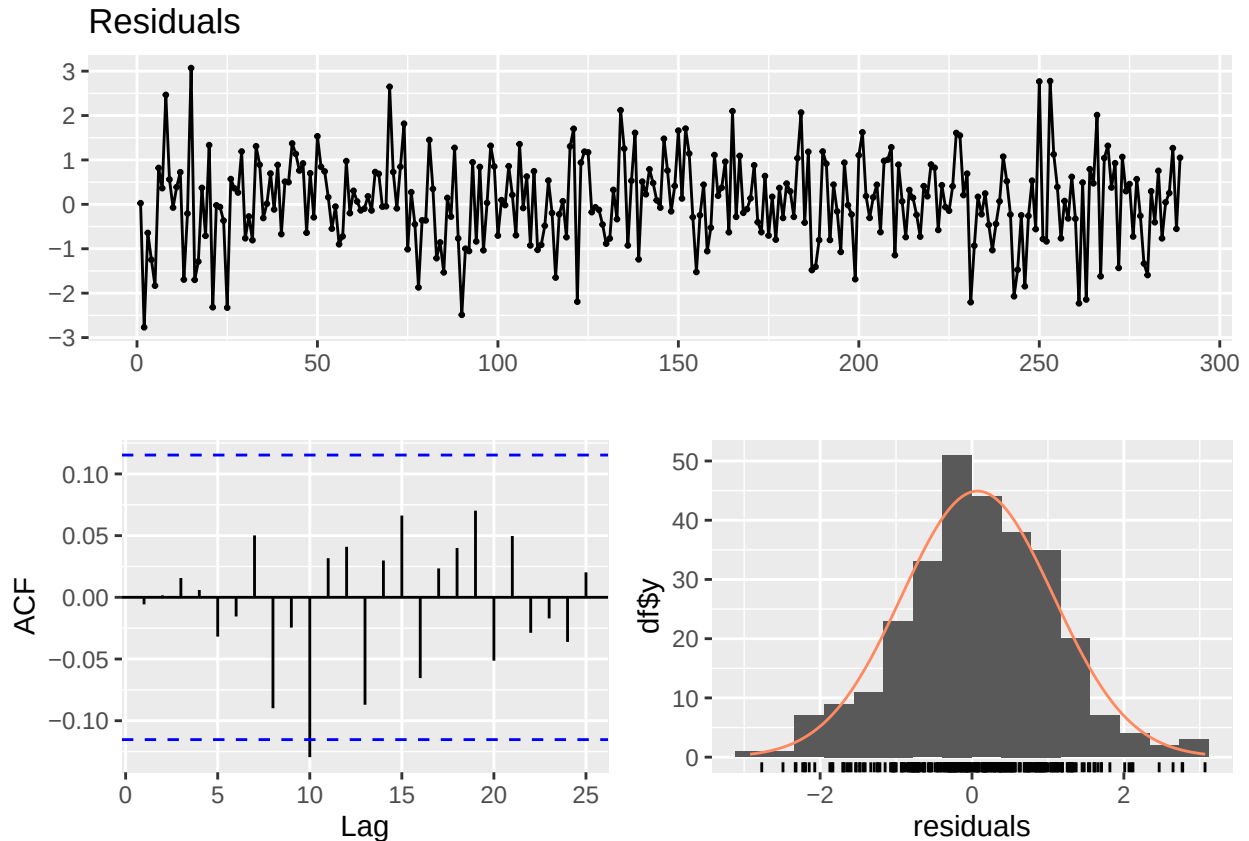
```
# plot each of components at once
plt_level <- plot_tempssm_components(res_ar1, component = c("level"))
plt_drift <- plot_tempssm_components(res_ar1, component = c("drift"))
plt_season <- plot_tempssm_components(res_ar1, component = c("season"))
plt_ar <- plot_tempssm_components(res_ar1, component = c("ar"))
```

Model Diagnostics

The package provides diagnostic tools for checking whether the fitted model has left notable structure in the residuals. In particular, residual time-series plots, residual autocorrelation, residual distributions, and Ljung-Box tests can be used to assess remaining temporal dependence and departures from the Gaussian error assumption.

```
r <- get_tempssm_residuals(res_ar1)
lb_lag <- frequency(res_ar1$temp_data)
```

```
forecast::checkresiduals(r, lag = lb_lag, test = "LB")
```



```
##
##  Ljung-Box test
##
## data:  Residuals
## Q* = 9.6827, df = 12, p-value = 0.6438
##
## Model df: 0.   Total lags used: 12
```

Here, `get_tempsm_residuals()` extracts standardized recursive residuals, and `forecast::checkresiduals()` displays their time-series plot, ACF plot, histogram, and Ljung-Box test. The three plots appear in the upper, lower-left, and lower-right panels, respectively. The lag is set to the seasonal frequency of the input data; for monthly data, this uses lag 12. These plots and the test result should be checked for any notable residual patterns.

In this example, the ACF plot does not show a clear sequence of successive significant lags or a repeated seasonal pattern. Isolated spikes can occur when many lags are inspected, so they should be interpreted together with the Ljung-Box test and the residual time-series plot rather than used as the sole basis for changing the model.

The package wrapper `plot_tempsm_residual_diagnostics()` is also available when a single function call for these diagnostic plots is preferred.

For a compact tabular summary of the Ljung-Box test and residual kurtosis, use `diagnose_residuals()`.

```
diag <- diagnose_residuals(res_ar1)
print(diag)
```

```
## # A tibble: 1 x 4
##   lb_stat lb_lag lb_pvalue kurtosis
##   <dbl> <dbl>   <dbl>   <dbl>
## 1    9.68    12    0.644    3.26
```

The `lb_stat`, `lb_lag`, and `lb_pvalue` columns returned by `diagnose_residuals()` correspond to the Ljung-Box test statistic, the lag used in the test, and the corresponding P-value. For monthly time series, the default Ljung-Box lag is the seasonal frequency, so the default test evaluates residual autocorrelation up to lag 12. In this example, the Ljung-Box test does not indicate significant residual autocorrelation up to lag 12.

If residual autocorrelation at longer lags is of concern, the lag can be specified manually. For example, the following code repeats the Ljung-Box test up to lags 24 and 36.

```
diag_lag24 <- diagnose_residuals(res_ar1, lb_lag = 24)
diag_lag36 <- diagnose_residuals(res_ar1, lb_lag = 36)

diag_lags <- rbind(diag, diag_lag24, diag_lag36)
diag_lags$check <- c("lag 12", "lag 24", "lag 36")
diag_lags[, c("check", "lb_stat", "lb_lag", "lb_pvalue", "kurtosis")]
```

```
## # A tibble: 3 x 5
##   check lb_stat lb_lag lb_pvalue kurtosis
##   <chr>   <dbl> <dbl>   <dbl>   <dbl>
## 1 lag 12    9.68    12    0.644    3.26
## 2 lag 24   19.5     24    0.725    3.26
## 3 lag 36   27.9     36    0.831    3.26
```

These diagnostics should be interpreted together with the residual ACF plot. A small number of isolated ACF spikes can occur by chance when many lags are inspected, whereas repeated or periodic spikes may suggest remaining temporal structure.

Examining the Autoregressive (AR) Order

In this manual, the default AR(1) specification is used as the working model. The autoregressive component is included to absorb short-term serial dependence that is not represented by the latent level, drift, and seasonal components. A higher AR order should therefore not be treated as an automatic improvement. Instead, the AR order can be adjusted when residual diagnostics suggest that notable autocorrelation remains.

For example, if residual autocorrelation remains after fitting the AR(1) model, users can fit an AR(2) model and repeat the same diagnostics.

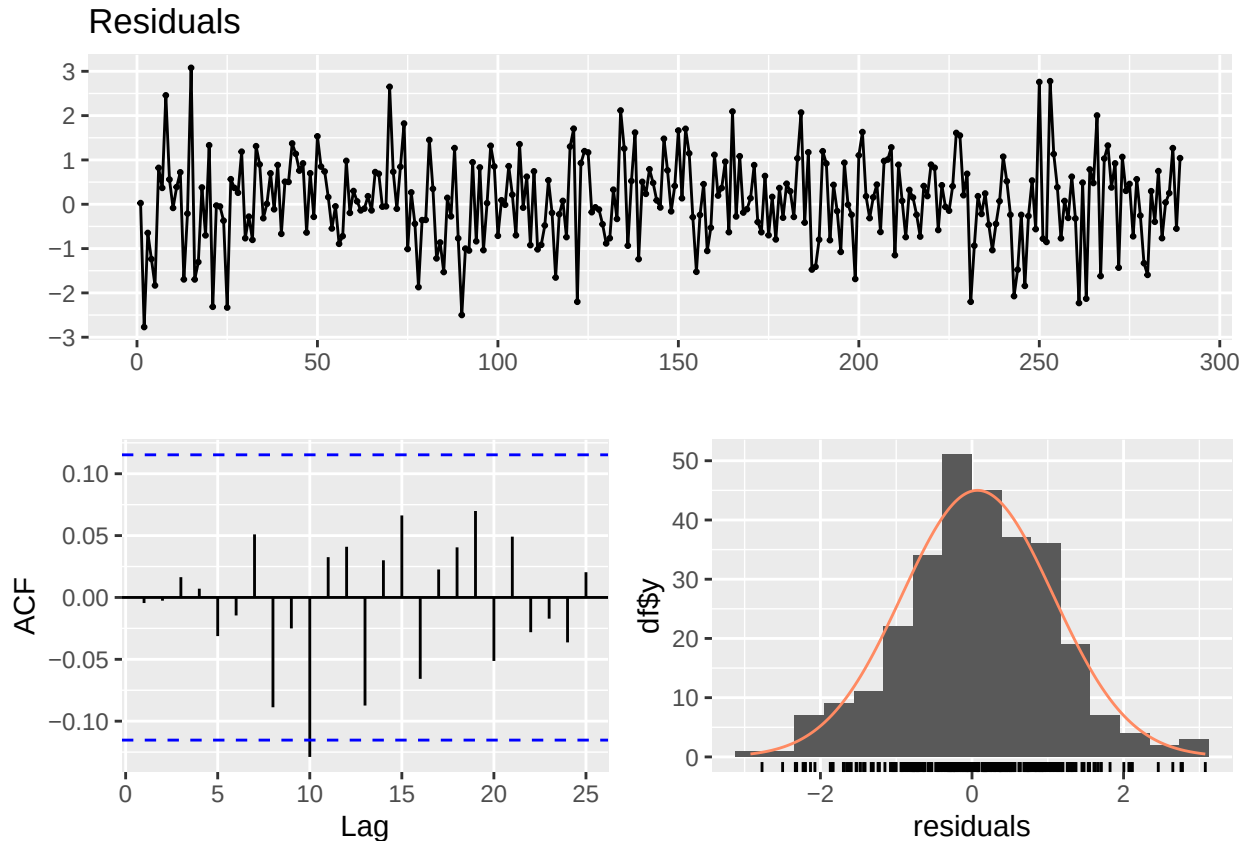
```
# Optional sensitivity check with a second-order autoregressive component
res_ar2 <- tempssm(sst_sim, ar_order = 2)
summary(res_ar2)
```

```
## tempssm summary
## -----
## Call:
## tempssm(temp_data = sst_sim, ar_order = 2)
##
## Model fit:
##   Likelihood type: marginal
##   Log-likelihood : -198.19
##   k               : 6
##   Diffuse states : 13
##   Converged      : TRUE
##
## Variance parameters:
```



```
## Observation (H): 0.05517452
## State (Q trend): 4.945287e-06
## State (Q season): 0.0001859987
## State (Q ar): 0.149755
##
## Components of auto-regression:
## Order of AR: 2
## Coefficient of AR1: 0.6553237
## Coefficient of AR2: 0.07430046
```

```
plot_tempssm_residual_diagnostics(res_ar2)
```



```
diag_lag12_ar2 <- diagnose_residuals(res_ar2, lb_lag = 12)
diag_lag24_ar2 <- diagnose_residuals(res_ar2, lb_lag = 24)
diag_lag36_ar2 <- diagnose_residuals(res_ar2, lb_lag = 36)
diag_lags_ar2 <- rbind(diag_lag12_ar2, diag_lag24_ar2, diag_lag36_ar2)
diag_lags_ar2$check <- c("lag 12", "lag 24", "lag 36")
diag_lags_ar2[, c("check", "lb_stat", "lb_lag", "lb_pvalue", "kurtosis")]
```

```
## # A tibble: 3 x 5
##   check lb_stat lb_lag lb_pvalue kurtosis
##   <chr>   <dbl> <dbl>   <dbl>   <dbl>
## 1 lag 12    9.61    12    0.650    3.26
## 2 lag 24   19.4    24    0.729    3.26
## 3 lag 36   27.9    36    0.831    3.26
```

In a sensitivity check with an AR(2) specification, the residual diagnostics were very similar to those from the AR(1) model. This suggests that increasing the AR order does not provide an obvious diagnostic improvement

in this example.

In the present example, the residual diagnostics do not provide strong evidence that a higher AR order is needed, so we proceed with the AR(1) model as the baseline specification.

Estimated Parameters and Latent-State Components

The fitted `tempssm` object contains the smoothed state estimates returned by KFAS. The matrix `res_ar1$kfs$alphahat` stores these latent-state estimates, including the level, drift, seasonal states, and autoregressive state. Inspecting this matrix is useful for understanding the internal state representation.

```
# Smoothing estimates
alpha_hat <- res_ar1$kfs$alphahat
head(alpha_hat)

##           level      slope sea_dumy1 sea_dumy2 sea_dumy3 sea_dumy4
## Jan 1998 18.65840 -0.01905354 -2.0428374 -0.9196344  0.5642451  0.9553992
## Feb 1998 18.63934 -0.01906722 -5.0256252 -2.0428374 -0.9196344  0.5642451
## Mar 1998 18.62028 -0.01909100 -2.8244535 -5.0256252 -2.0428374 -0.9196344
## Apr 1998 18.60118 -0.01911003 -2.0508949 -2.8244535 -5.0256252 -2.0428374
## May 1998 18.58207 -0.01914415  0.2882283 -2.0508949 -2.8244535 -5.0256252
## Jun 1998 18.56293 -0.01918604  1.9925949  0.2882283 -2.0508949 -2.8244535
##           sea_dumy5 sea_dumy6 sea_dumy7 sea_dumy8 sea_dumy9 sea_dumy10
## Jan 1998  1.6282639  4.9410126  2.4937014  1.9925949  0.2882283 -2.0508949
## Feb 1998  0.9553992  1.6282639  4.9410126  2.4937014  1.9925949  0.2882283
## Mar 1998  0.5642451  0.9553992  1.6282639  4.9410126  2.4937014  1.9925949
## Apr 1998 -0.9196344  0.5642451  0.9553992  1.6282639  4.9410126  2.4937014
## May 1998 -2.0428374 -0.9196344  0.5642451  0.9553992  1.6282639  4.9410126
## Jun 1998 -5.0256252 -2.0428374 -0.9196344  0.5642451  0.9553992  1.6282639
##           sea_dumy11      arima1
## Jan 1998 -2.8244535 -0.2178662
## Feb 1998 -2.0508949  0.1519895
## Mar 1998  0.2882283  0.4187224
## Apr 1998  1.9925949  0.2408084
## May 1998  2.4937014  0.7185730
## Jun 1998  4.9410126  1.0167731
```

For routine use, helper functions provide a simpler way to extract individual components as `ts` objects with the original time index. The level component represents the estimated long-term temperature level, while the drift component represents its rate of change per year.

```
# Smoothing estimate of level component
level_ts <- get_level_ts(res_ar1)

# Smoothing estimate of drift component
drift_ts <- get_drift_ts(res_ar1)

# Average drift rate per year across the full period
mean_drift_year <- mean(drift_ts)
print(mean_drift_year)
```

```
## [1] 0.08777084
```

The average annual rate of SST change was estimated to be 0.0878 °C over the full period. Because the estimated drift changes sign over time, this value should be interpreted as a model-based summary of the whole study period rather than as a constant warming rate.

Short-Term Prediction

A fitted `tempssm` object can also be passed to `predict()` to obtain short-term predictions. By default, `predict(res)` returns a one-step-ahead prediction beyond the end of the observed series. This is useful for visual checks of how the fitted model extrapolates the estimated level, seasonal, and autoregressive components.

```
pred_1 <- predict(res_ar1)
pred_1
```

```
##           Mar
## 2023 18.33035
```

Predictions for multiple future time points can be requested by setting the `n.ahead` argument.

```
pred_12 <- predict(res_ar1, n.ahead = 12)
pred_12
```

```
##           Jan           Feb           Mar           Apr           May           Jun           Jul           Aug
## 2023                18.33035 18.96985 21.33710 23.08522 23.51856 26.03694
## 2024 19.09319 16.16926
##           Sep           Oct           Nov           Dec
## 2023 22.69098 22.00902 21.74806 20.19190
## 2024
```

Prediction uncertainty can also be returned by setting the `interval` argument. With `interval = "confidence"`, the returned lower and upper bounds describe uncertainty in the predicted mean response. With `interval = "prediction"`, the bounds describe uncertainty in a future observation and therefore also include observation error; prediction intervals are usually wider than confidence intervals. The confidence level is controlled by the `level` argument and defaults to 0.95.

```
pred_12_pi <- predict(
  res_ar1,
  n.ahead = 12,
  interval = "prediction",
  level = 0.95
)
```

```
pred_12_pi
```

```
##           fit           lwr           upr
## Mar 2023 18.33035 17.35029 19.31042
## Apr 2023 18.96985 17.85360 20.08610
## May 2023 21.33710 20.13453 22.53967
## Jun 2023 23.08522 21.82425 24.34619
## Jul 2023 23.51856 22.21551 24.82161
## Aug 2023 26.03694 24.70183 27.37204
## Sep 2023 22.69098 21.33016 24.05179
## Oct 2023 22.00902 20.62664 23.39141
## Nov 2023 21.74806 20.34682 23.14929
## Dec 2023 20.19190 18.77359 21.61020
## Jan 2024 19.09319 17.65904 20.52734
## Feb 2024 16.16926 14.72135 17.61717
```

These predictions should be interpreted as model-based extrapolations rather than definitive forecasts. Uncertainty generally increases as the prediction horizon becomes longer, and long-horizon predictions can be sensitive to model assumptions about trend, seasonality, and autoregressive dependence.

Exercise II: Model With Exogenous Variables

Objective

We extend the state-space modeling framework to examine the effect of an exogenous factor on temperature variation. Specifically, we examine simulated Kuroshio path information as exogenous variables for the simulated SST series introduced in Exercise I.

Loading Simulated Kuroshio Exogenous Variables

Two simulated exogenous-variable datasets are included in the package.

- **kuroshio_a_sim**: monthly binary indicator of the Kuroshio A-type path
- **distance_sim**: monthly offshore-distance index of the Kuroshio path
- **Period**: January 1998 to February 2023

Both datasets are provided as univariate **ts** objects with **frequency = 12**. They are simulated monthly datasets based on the Kuroshio-related variables used in Baba et al. (2024). The original data used in that study had finer time resolution for some variables, but the simulated datasets in this package are represented as monthly time series for use with **tempssm** examples.

```
data(kuroshio_a_sim)
```

```
data(distance_sim)
```

```
head(kuroshio_a_sim)
```

```
##      Jan Feb Mar Apr May Jun
## 1998   0   0   0   0   0   0
```

```
head(distance_sim)
```

```
##      Jan Feb Mar Apr May Jun
## 1998 116 148 49 131 45 40
```

```
summary(kuroshio_a_sim)
```

```
##      kuroshio_a
## Min.      :0.0000
## 1st Qu.:0.0000
## Median :0.0000
## Mean    :0.2483
## 3rd Qu.:0.0000
## Max.    :1.0000
```

```
summary(distance_sim)
```

```
##      distance
## Min.      : 10.00
## 1st Qu.: 26.00
## Median : 43.00
## Mean    : 55.65
## 3rd Qu.: 84.00
## Max.    :150.00
```

Checking Time-Series Alignment

For state-space modeling with exogenous variables, the response and exogenous time series must share a common and aligned time index. In this example, **sst_sim**, **kuroshio_a_sim**, and **distance_sim** already

have the same monthly time span and frequency.

```
series_info <- tibble::tibble(
  series = c("sst_sim", "kuroshio_a_sim", "distance_sim"),
  start = c(
    paste(start(sst_sim), collapse = "-"),
    paste(start(kuroshio_a_sim), collapse = "-"),
    paste(start(distance_sim), collapse = "-")
  ),
  end = c(
    paste(end(sst_sim), collapse = "-"),
    paste(end(kuroshio_a_sim), collapse = "-"),
    paste(end(distance_sim), collapse = "-")
  ),
  frequency = c(
    frequency(sst_sim),
    frequency(kuroshio_a_sim),
    frequency(distance_sim)
  ),
  missing = c(
    sum(is.na(sst_sim)),
    sum(is.na(kuroshio_a_sim)),
    sum(is.na(distance_sim))
  )
)

series_info
```

```
## # A tibble: 3 x 5
##   series      start end   frequency missing
##   <chr>      <chr> <chr>         <dbl>    <int>
## 1 sst_sim    1998-1 2023-2         12        0
## 2 kuroshio_a_sim 1998-1 2023-2         12        0
## 3 distance_sim 1998-1 2023-2         12        0
```

If user-supplied time series have different start or end points, `trim_ts_overlap()` can be used to restrict the response and exogenous series to their shared overlapping period before fitting the model.

Plotting the Response and Exogenous Variables

We visualize the simulated SST series together with the two simulated Kuroshio variables to inspect the overall structure of the dataset.

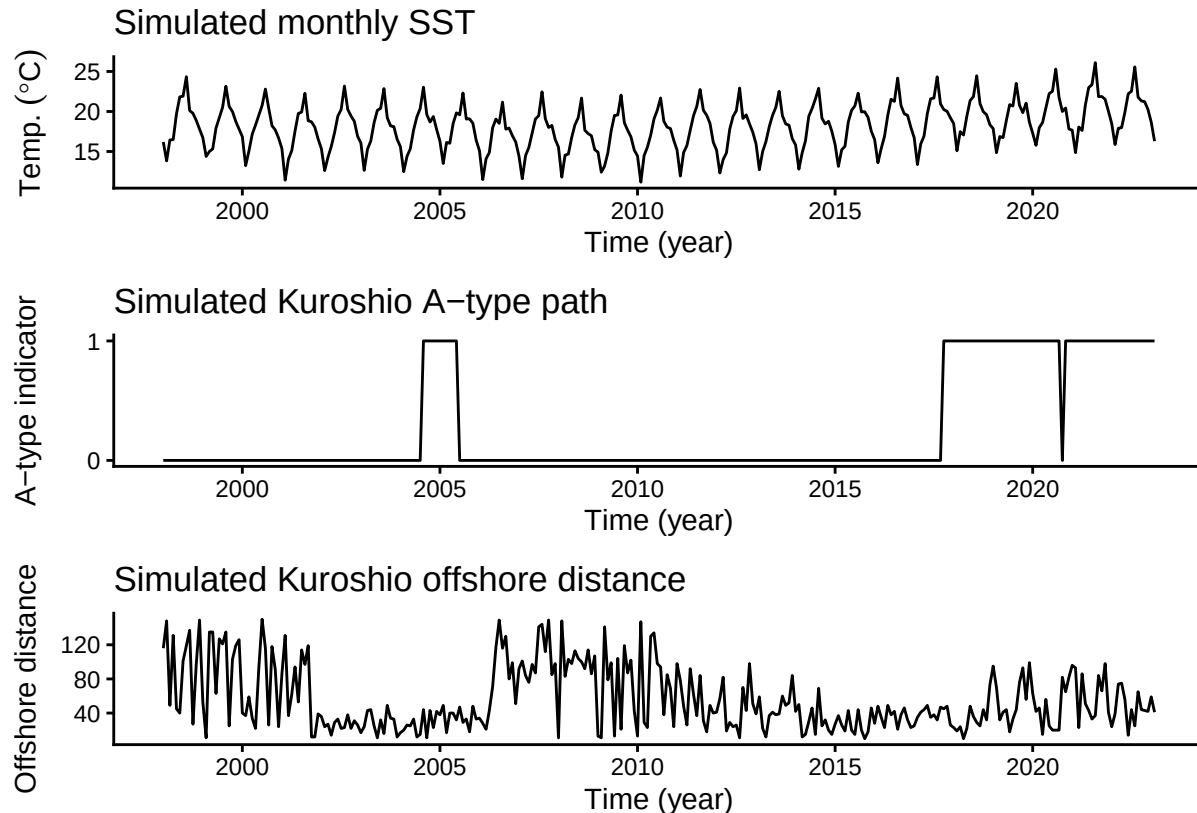
```
plt_sst_exo <- forecast::autoplot(sst_sim) +
  labs(y = expression(Temp.~(degree*C)), x = "Time (year)") +
  ggtitle("Simulated monthly SST") +
  theme_classic()

plt_kuroshio_a <- forecast::autoplot(kuroshio_a_sim) +
  labs(x = "Time (year)", y = "A-type indicator") +
  ggtitle("Simulated Kuroshio A-type path") +
  scale_y_continuous(breaks = c(0, 1)) +
  theme_classic()

plt_distance <- forecast::autoplot(distance_sim) +
  labs(x = "Time (year)", y = "Offshore distance") +
```

```
ggtitle("Simulated Kuroshio offshore distance") +
theme_classic()

plt_sst_exo + plt_kuroshio_a + plt_distance +
patchwork::plot_layout(ncol = 1)
```



The simulated exogenous-variable series used in this exercise contain no missing values. When using your own exogenous-variable data, make sure that the exogenous variables do not contain missing values before fitting the model. `tempssm()` does not allow missing values in exogenous variables, and model fitting will stop with an error if such data are supplied. If necessary, apply appropriate preprocessing, such as imputation, before the analysis.

In contrast, missing values in the dependent temperature time series are allowed. In the state-space model, these missing values are treated as unobserved responses, and model estimation proceeds through Kalman filtering and smoothing.

Applying a Model Without an Exogenous Variable

The baseline model without exogenous variables was already fitted in Exercise I. Because Exercise II uses the same response series, `sst_sim`, we reuse that fitted object as the reference model instead of refitting the same model.

```
res_without <- res_ar1
summary(res_without)

## tempssm summary
## -----
## Call:
## tempssm(temp_data = sst_sim)
```

```
##
## Model fit:
##   Likelihood type: marginal
##   Log-likelihood : -198.19
##   k               : 5
##   Diffuse states : 13
##   Converged       : TRUE
##
## Variance parameters:
##   Observation (H): 0.07496416
##   State (Q trend): 4.937122e-06
##   State (Q season): 0.0001763538
##   State (Q ar): 0.1202156
##
## Components of auto-regression:
##   Order of AR: 1
##   Coefficient of AR1: 0.7579372
```

This avoids repeating the same estimation step and makes the comparison focus on the additional information introduced by the simulated Kuroshio variables.

Combining Exogenous Variables into a Multivariate `ts` Object

The `exo_data` argument of `tempssm()` can accept either a univariate `ts` object or a multivariate `ts` object. Because this exercise uses two exogenous variables, we combine `kuroshio_a_sim` and `distance_sim` into a single multivariate `ts` object before fitting the model.

```
exo_kuroshio <- cbind(kuroshio_a_sim, distance_sim)
colnames(exo_kuroshio) <- c("kuroshio_a", "distance")

head(exo_kuroshio)
```

```
##           kuroshio_a distance
## Jan 1998           0      116
## Feb 1998           0      148
## Mar 1998           0       49
## Apr 1998           0      131
## May 1998           0       45
## Jun 1998           0       40
```

```
start(exo_kuroshio)
```

```
## [1] 1998      1
```

```
end(exo_kuroshio)
```

```
## [1] 2023      2
```

```
frequency(exo_kuroshio)
```

```
## [1] 12
```

```
colnames(exo_kuroshio)
```

```
## [1] "kuroshio_a" "distance"
```

```
sum(is.na(exo_kuroshio))
```

```
## [1] 0
```

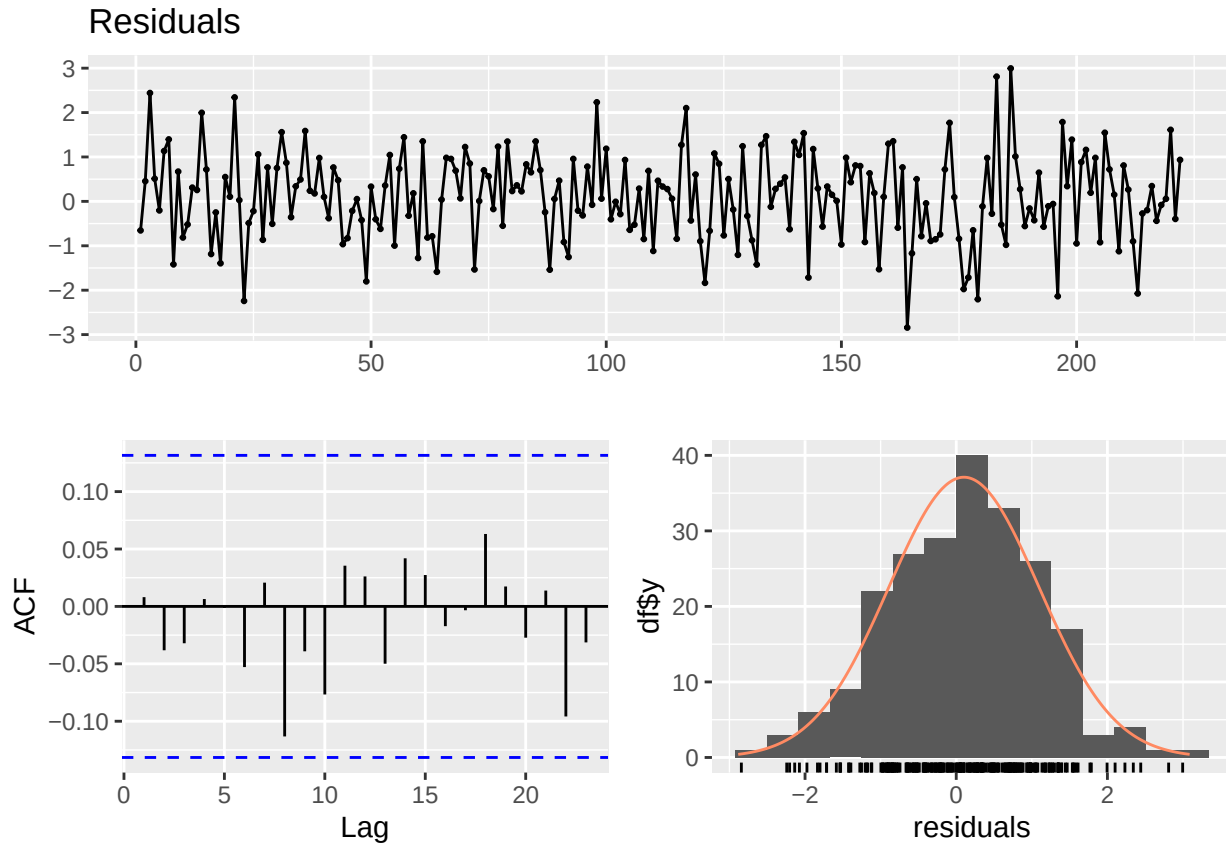
The resulting object has class `mts`, which is the multivariate form of base R's `ts` class. Each column is treated as a separate exogenous variable in the state-space model. The column names are used as variable labels in summaries and coefficient tables, so assigning informative names before model fitting is helpful.

Applying a Model With Exogenous Variables

We next construct a model that uses the two simulated Kuroshio variables as exogenous variables. In the `tempssm()` function, the multivariate object `exo_kuroshio` is supplied to the exogenous-variable argument (`exo_data`), and the simulated SST series (`sst_sim`) is supplied to the dependent-variable argument (`temp_data`).

```
res_with <- tempssm(
  temp_data = sst_sim,
  exo_data = exo_kuroshio,
  ar_order = 1
)
summary(res_with)

## tempssm summary
## -----
## Call:
## tempssm(temp_data = sst_sim, exo_data = exo_kuroshio, ar_order = 1)
##
## Model fit:
##   Likelihood type: marginal
##   Log-likelihood : -154.38
##   k               : 7
##   Diffuse states : 15
##   Converged      : TRUE
##
## Variance parameters:
##   Observation (H): 0.003184407
##   State (Q trend): 4.132733e-06
##   State (Q season): 0.0006818023
##   State (Q ar): 0.1556755
##
## Components of auto-regression:
##   Order of AR: 1
##   Coefficient of AR1: 0.6585956
## Exogenous variable   kuroshio_a distance
## Estimated coefficient    0.5110124 -0.007070968
## Lower CI   0.1658023 -0.008467629
## Upper CI   0.8562225 -0.005674308
plot_tempssm_residual_diagnostics(res_with)
```

```
diag_res_with <- diagnose_residuals(res_with)
print(diag_res_with)
```

```
## # A tibble: 1 x 4
##   lb_stat lb_lag lb_pvalue kurtosis
##   <dbl>   <dbl>   <dbl>   <dbl>
## 1     6.50     12     0.889     3.07
```

The residual diagnostics for the model with the simulated Kuroshio variables also do not indicate significant residual autocorrelation up to lag 12. Thus, for both models considered here, the AR(1) specification provides an adequate working autoregressive structure according to these residual checks.

We then examine the estimated coefficients for the exogenous variables.

```
exo_coef

##   Variable Coefficient      lwr      upr
## 1 kuroshio_a 0.511012406 0.165802340 0.856222472
## 2 distance -0.007070968 -0.008467629 -0.005674308
```

The coefficient table summarizes the estimated associations between SST and each simulated Kuroshio variable after accounting for the latent long-term trend, seasonal cycle, and autoregressive dependence. Because these are simulated example datasets, the estimates should be interpreted as an illustration of how to fit and inspect a model with multiple exogenous variables, rather than as new scientific evidence about the Kuroshio path.

For models with exogenous variables, future values of the exogenous variables are needed when calling `predict()`. If such values are available, they can be passed through `new_exo_data`. For a model with multiple exogenous variables, `new_exo_data` should contain the same variables, in the same order, as the object supplied to `exo_data`. For a simple one-step-ahead visual check, `exo_strategy = "last"` carries the final observed row of all exogenous variables forward. This is a persistence assumption for the exogenous

variables, not a separate forecast model for the Kuroshio path.

```
pred_with_last_exo <- predict(res_with, exo_strategy = "last")
pred_with_last_exo
```

```
##           Mar
## 2023 18.37914
```

The next section evaluates whether including these simulated Kuroshio variables improves out-of-sample predictive performance under a conditional forecasting setting.

Time-Series Cross-Validation (tsCV)

Time-series cross-validation (tsCV) evaluates model performance based on out-of-sample prediction errors while respecting the temporal ordering of the data. It therefore provides an additional and independent perspective on model adequacy.

In time-series cross-validation, the model is repeatedly fitted to expanding or sliding training windows, and predictive performance is evaluated on subsequent observations. Unlike random cross-validation, this procedure prevents information leakage from the future to the past and is therefore suitable for model validation with temporal data.

Here, by comparing cross-validation metrics for models with and without the simulated Kuroshio variables, we assess whether the additional exogenous information is reflected in out-of-sample predictive performance.

For the model with the simulated Kuroshio variables, the test-period values of those variables are supplied to the forecast step as exogenous variables. Therefore, this tsCV setting evaluates conditional forecasts, or hindcasts, under the assumption that the Kuroshio-variable values during each test period are known. This is appropriate when the aim is to assess the explanatory or reconstructive value of the exogenous variables.

For real-time future forecasting, however, future exogenous-variable values are usually unknown. In that setting, users must either provide future scenarios, forecast the exogenous variables with separate models, or use a simplified assumption such as carrying the last observed value forward. The results below should therefore be interpreted as evidence for improved conditional predictive performance, not as a direct guarantee of operational forecast improvement.

```
# (Optional) Load packages for parallel processing.
# These are only required if you want to enable parallel execution:
# - future: controls how parallel workers are launched
# - future.apply: provides parallelized apply functions
# - progressr: optional progress bar support
library(future.apply)
library(future)
library(progressr)

if (interactive()) {
  future::plan(multisession)
} else {
  future::plan(sequential)
}

## Generate a list of training and test datasets with their indices
# Procedure for constructing year-based time-series cross-validation folds:
#
# First training data: January 1998-December 2010;
# First test data: January 2011-December 2011
#
```

```

# Second training data: January 1999–December 2011;
# Second test data: January 2012–December 2012
# ...
#
# Twelfth training data: January 2009–December 2021;
# Twelfth test data: January 2022–December 2022
#
# These folds are automatically generated by the ts_train_test_split() function.
# The final two observations in 2023 are not used because allow_partial = FALSE
# keeps every test set as a complete 12-month period.

# Generate training and test dataset for model without exogenous variables
folds_without <- ts_train_test_split(
  temp_data = sst_sim,
  exo_data = NULL,
  initial = 156, # 156 monthly observations from Jan 1998 to Dec 2010
  horizon = 12, # forecast 12 monthly observations
  step = 12, # move the fixed-width window in one-year steps
  fixed_window = TRUE,
  allow_partial = FALSE
)

# Generate training and test dataset for model with Kuroshio variables
folds_with <- ts_train_test_split(
  temp_data = sst_sim,
  exo_data = exo_kuroshio,
  initial = 156, # 156 monthly observations from Jan 1998 to Dec 2010
  horizon = 12, # forecast 12 monthly observations
  step = 12, # move the fixed-width window in one-year steps
  fixed_window = TRUE,
  allow_partial = FALSE
)

# Check the first fold
start(folds_without[[1]]$train_ts)

## [1] 1998    1
end(folds_without[[1]]$train_ts)

## [1] 2010    12
start(folds_without[[1]]$test_ts)

## [1] 2011    1
end(folds_without[[1]]$test_ts)

## [1] 2011    12
# *****
# Executing time-series cross validation
#-----
# Model without exogenous variables
# Single processing

```

```

# cv_without_results <- ts_cv_run(
#   folds_without,
#   ar_order = 1,
#   use_season = TRUE
# )

# Parallel processing
cv_without_results <- future.apply::future_lapply(
  folds_without,
  ts_cv_run_fold,
  ar_order = 1,
  use_season = TRUE,
  future.seed = TRUE
)

# Computing assessment indexes
metrics_without <- lapply(cv_without_results, compute_cv_metrics)

# tidy summary
cv_without_tbl <- ts_cv_collect(cv_without_results, metrics_without) %>%
  mutate(Model="Without")

#-----
# Model with simulated Kuroshio exogenous variables

# Single processing
# cv_with_results <- ts_cv_run(folds_with, ar_order = 1, use_season = TRUE)

# Parallel processing
cv_with_results <- future.apply::future_lapply(
  folds_with,
  ts_cv_run_fold,
  ar_order = 1,
  use_season = TRUE,
  future.seed = TRUE
)

# Computing assessment indexes
metrics_with <- lapply(cv_with_results, compute_cv_metrics)

# tidy summary
cv_with_tbl <- ts_cv_collect(cv_with_results, metrics_with) %>%
  mutate(Model="With")

cv_tbl <- bind_rows(cv_without_tbl, cv_with_tbl)

cv_comparison <- compare_ts_cv(
  list(
    Without = cv_without_tbl,
    With = cv_with_tbl
  )
)

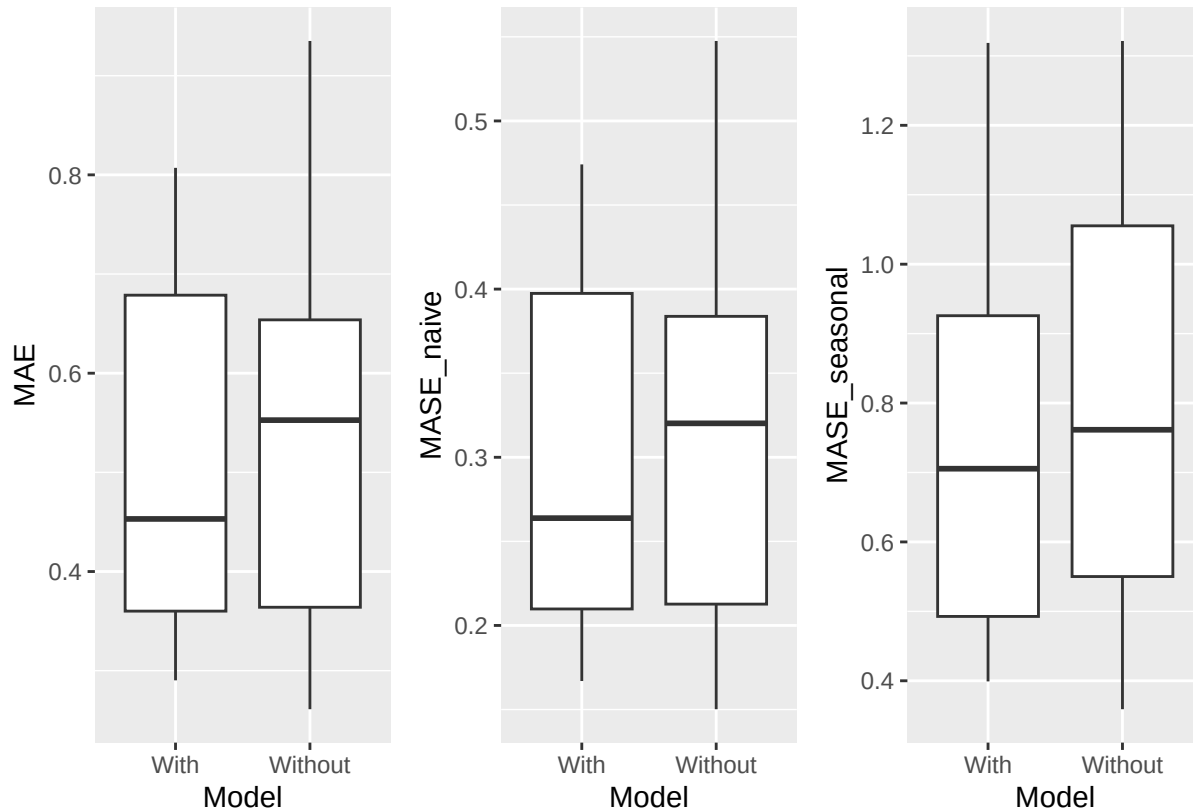
```

```
)
```

```
cv_comparison %>% knitr::kable()
```

model	n_folds	con- verged_n	converged_rate	mean_MAE	mean_MASE_naive	mean_MASE_seasonal
With-out	12	12	1	0.5375966	0.3137671	0.8001065
With	12	12	1	0.5135379	0.2997563	0.7705862

```
plt_MAE <- ggplot(data=cv_tbl,  
                  aes(x=Model,y=MAE)) +  
  geom_boxplot()  
  
plt_MASE_naive <- ggplot(data=cv_tbl,  
                         aes(x=Model,y=MASE_naive)) +  
  geom_boxplot()  
  
plt_MASE_seasonal <- ggplot(data=cv_tbl,  
                           aes(x=Model,y=MASE_seasonal)) +  
  geom_boxplot()  
  
plt_tsCV <- plt_MAE + plt_MASE_naive + plt_MASE_seasonal +  
  patchwork::plot_layout(nrow = 1)  
  
plot(plt_tsCV)
```



The tsCV results show that the model with the exogenous variable performs better than the model without it. The comparison table produced by `compare_ts_cv()` summarizes the number of folds, convergence rates, and mean prediction-error metrics for each model, while the boxplots show how the fold-level errors vary across validation periods. In this tutorial, the number of tsCV iterations is set to 12 to reduce execution time. In actual validation, please use a sufficient number of iterations.

The mean MAE of the model with the exogenous variable is approximately 0.51 °C. Relative to the seasonal amplitude based on monthly mean SST (about 9.9 °C), this corresponds to roughly 5.2%. Thus, the prediction error is small compared with the dominant seasonal scale of variation in this data set, and may be practically useful for short-term marine monitoring.

Although the number of validation folds is limited to 12, the model with the exogenous variable reduces the mean MAE by approximately 4.5% compared with the model without the exogenous variable. Similar improvements are also seen in the MASE metrics. This interpretation should be kept conditional on the tsCV setting used here, in which the test-period values of the simulated Kuroshio variables are supplied as exogenous variables.

Taken together, the results illustrate how exogenous-variable models can be evaluated using both coefficient estimates and time-series cross-validation. Because the datasets in this exercise are simulated, the emphasis is on the workflow rather than on drawing new scientific conclusions about Kuroshio variability.

These complementary lines of evidence can help users decide whether an exogenous-variable model is useful for their own data, while keeping the forecasting assumptions for future exogenous variables explicit.

This exercise used two exogenous variables supplied as multivariate time-series data. Users interested in broader applications can extend the same workflow to other environmental, climatic, or ecological covariates. In doing so, model structure should be considered carefully for the target data, while checking model diagnostics and out-of-sample predictive performance.

Comparing Estimation Patterns Between Models

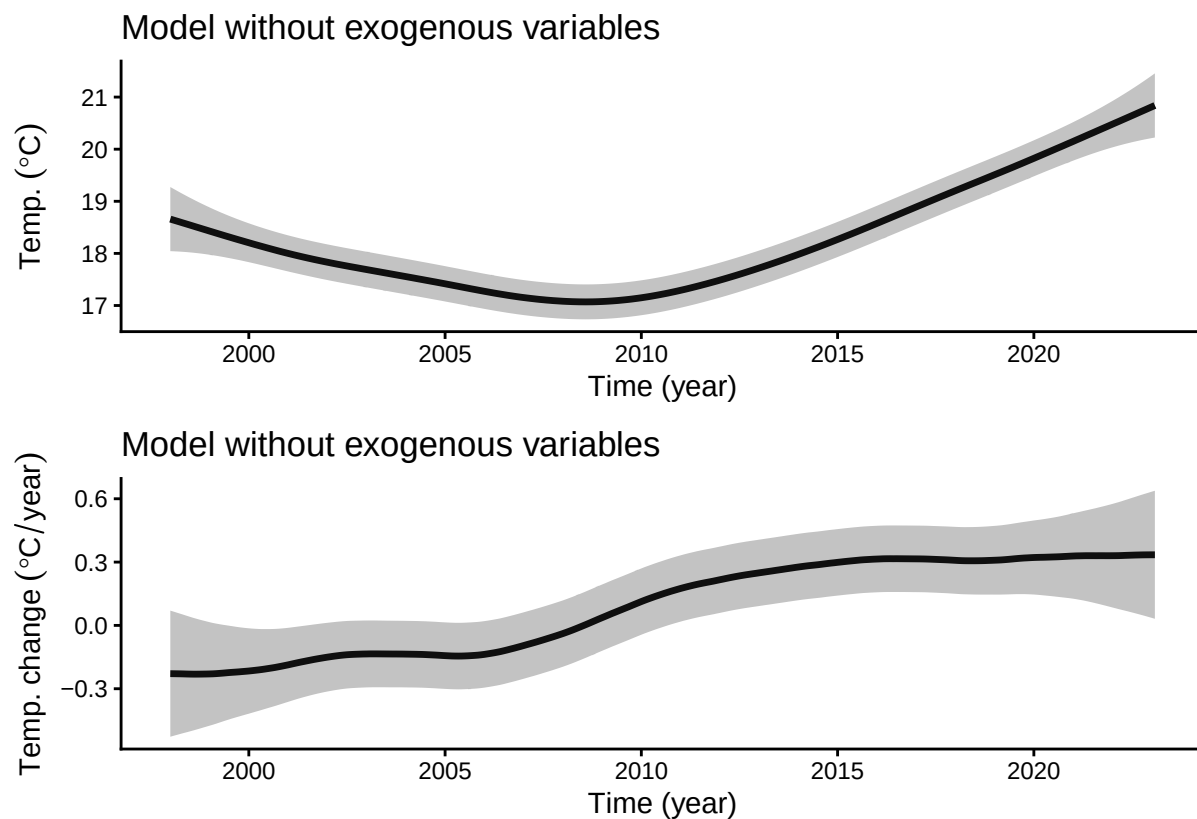
We compare how the estimated patterns of the long-term trend and its rate of change differ between the models with and without the simulated Kuroshio variables.

```
plt_level_without_ts <- autoplot(res_without,
                                component=c("level")
                                ) +
  labs(title="Model without exogenous variables") +
  theme_classic()

plt_drift_without_ts <- autoplot(res_without,
                                component=c("drift")
                                ) +
  labs(title="Model without exogenous variables") +
  theme_classic()

plt_level_drift_without_ts <- plt_level_without_ts +
  plt_drift_without_ts +
  patchwork::plot_layout(ncol=1)

plot(plt_level_drift_without_ts)
```

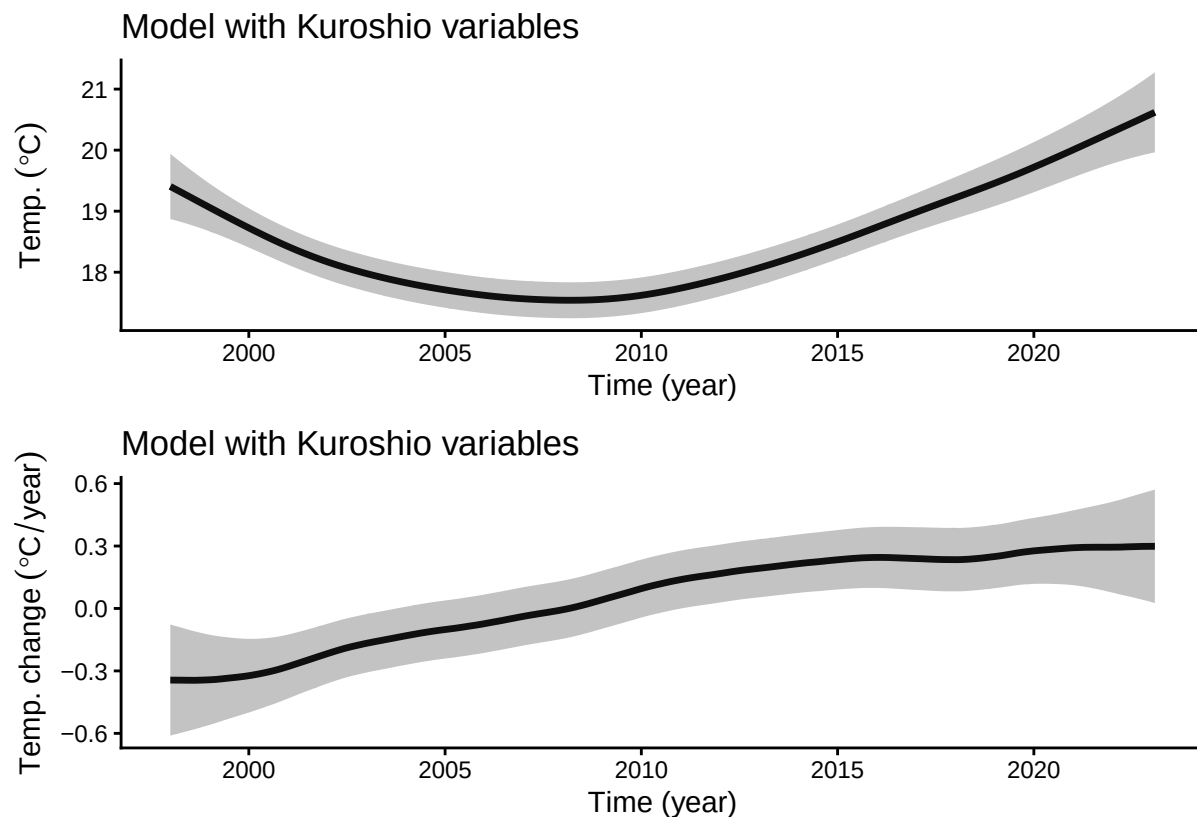


```
plt_level_with_ts <- autoplot(res_with,
                              component=c("level")
                              ) +
  labs(title="Model with Kuroshio variables") +
  theme_classic()
```

```
plt_drift_with_ts <- autoplot(res_with,
                             component=c("drift")
                             ) +
  labs(title="Model with Kuroshio variables") +
  theme_classic()

plt_level_drift_with_ts <- plt_level_with_ts +
  plt_drift_with_ts +
  patchwork::plot_layout(ncol=1)

plot(plt_level_drift_with_ts)
```



```
# Smoothing estimate of drift component
mean_drift_year_without <- get_drift_ts(res_without) %>%
  mean()
print(mean_drift_year_without)
```

```
## [1] 0.08777084
```

```
mean_drift_year_with <- get_drift_ts(res_with) %>%
  mean()
print(mean_drift_year_with)
```

```
## [1] 0.04924515
```

The gray areas in the plots above show 95% confidence intervals.

In the model without an exogenous variable, the plots of the long-term trend and its rate of change show a wavelike pattern. In contrast, in the model with the exogenous variable, the long-term trend and its rate of change show a more stable pattern. This suggests that the wavelike pattern reflects the influence of variation

represented by the simulated Kuroshio variables.

The estimated annual rate of SST change over the observation period is 0.0878 in the model without the exogenous variable and 0.0492 in the model with the exogenous variable.

In this simulated example, the model incorporating the exogenous variables provides an alternative decomposition of the SST series after accounting for the additional covariates. The comparison illustrates that estimated long-term components can change when external drivers are included, which is one reason to inspect both model diagnostics and predictive performance before selecting a working model.

Using Your Own Data

Quick Example

`my_ts` is a user-supplied `ts` object containing temperature time-series data. It can be passed directly to `tempssm()`.

```
## example of ts object (not run)
## my_ts <- ts(rnorm(100), frequency = 12)
res <- tempssm(my_ts)
```

Preparing Your Own Dataset

CSV Format

Prepare a CSV file of monthly temperature time series with the following columns: - Year - Month - Temp

Example:

```
Year,Month,Temp
2010,8,13.6
2010,9,6.8
2010,10,NA
2010,11,-1.4
...
```

- Use NA for missing temperature values, and always keep the corresponding Year and Month entries.
- The CSV file must be comma-separated and UTF-8 encoded.

Converting CSV to a Time Series

An example CSV file included in this package is available in `inst/extdata`. The example dataset contains monthly air temperature observations from Mt. Akadake, Hokkaido, Japan (1,840 m). The original data are available from the Monitoring Sites 1000 Project of the Ministry of the Environment of Japan (KOZ01.zip, <https://www.biodic.go.jp/moni1000/findings/data/index.html>).

```
path <- system.file("extdata", "example_monthly_temp.csv", package = "tempssm")
akadake_temp <- tempssm::read_monthly_temp_ts(path)
head(akadake_temp)
```

```
##           Jan Feb Mar Apr May Jun Jul   Aug   Sep   Oct   Nov   Dec
## 2010                13.6   6.8   0.2  -6.8 -12.5
## 2011 -18.8
```

The `read_monthly_temp_ts()` function reads this type of CSV file and converts it into an R `ts` object for use with `tempssm()`. If finer control is needed, you can also construct a `ts` object manually; see the `stats::ts` documentation for details: <https://search.r-project.org/R/refmans/stats/html/ts.html>.

References

The statistical modeling framework implemented in **tempssm** is based on the methodology described in Baba et al. (2024), and the accompanying supplementary materials and code repository served as the initial source for development of this package.

Baba, S., Ishii, H., and Yoshiyama, T. (2024). Estimating sea temperature trends using a linear Gaussian state-space model in Jogashima, Kanagawa, Japan. *Bulletin of the Japanese Society of Fisheries Oceanography*, 88(3), 190–199. (In Japanese with an English abstract.) https://doi.org/10.34423/jsfo.88.3_190

Baba, S. (2024). Supplementary code and test data for estimating sea temperature trends using a linear Gaussian state-space model. GitHub repository: <https://github.com/logics-of-blue/sea-temperature-trend-jogashima>

Helske, J. (2017). KFAS: Exponential Family State Space Models in R. *Journal of Statistical Software*, 78(10), 1–39. <https://doi.org/10.18637/jss.v078.i10>

Appendix: Utility Functions

The following utility functions are provided to support data preparation and exploratory analysis.

1. `read_monthly_temp_ts()`

The `read_monthly_temp_ts()` function converts monthly temperature data stored in a CSV file into an R `ts` object. By enforcing a simple and consistent data format, it is intended to make externally prepared time-series data easier to import into **tempssm**.

Example

For monthly temperature data, prepare a CSV file with a header row. By default, the column names must be `Year`, `Month`, and `Temp`, where `Temp` represents the observed temperature value.

```
Year,Month,Temp
2001,1,10.4
2001,2,8.2
2001,3,NA
2001,4,13.6
2001,5,16.1
...
```

- Use NA for missing temperature values, and always keep the corresponding Year and Month entries.
- The CSV file must be comma-separated and UTF-8 encoded.

The following example uses the sample CSV file included in the package. It contains monthly air temperature observations from Mt. Akadake, Hokkaido, Japan.

```
path <- system.file("extdata", "example_monthly_temp.csv", package = "tempssm")

# Read the CSV file and convert it to a monthly ts object
temp_ts <- read_monthly_temp_ts(path)
head(temp_ts)
```

```
##           Jan Feb Mar Apr May Jun Jul   Aug   Sep   Oct   Nov   Dec
## 2010                                13.6   6.8   0.2  -6.8 -12.5
## 2011 -18.8
```

2. convert_monthly_df_to_ts()

The `convert_monthly_df_to_ts()` function converts a data frame containing monthly temperature data into an R `ts` object. It is designed to support workflows in which temperature data are first imported or prepared as a data frame and then used for time-series analysis.

The input data frame is expected to contain at least two columns: a date column (`Date`) and a temperature column (`Temp`). For the date column (`Date`), assign an arbitrary day of the month, such as the first day, so that the dates correspond to monthly aggregated temperature values.

Example

```
# Create a data frame of monthly temperature data
df <- data.frame(
  Date = as.Date(c(
    "2001-01-01",
    "2001-02-01",
    "2001-03-01",
    "2001-04-01",
    "2001-05-01")
  ),
  Temp = c(10.4, 8.2, NA, 13.6, 16.1)
)

# Convert to a monthly ts object
temp_ts <- convert_monthly_df_to_ts(df)
head(temp_ts)
```

```
##           Jan  Feb  Mar  Apr  May
## 2001 10.4   8.2   NA 13.6 16.1
```

3. get_jma_sst_ts()

The `get_jma_sst_ts()` function downloads daily mean sea surface temperature (SST) data for Japanese coastal waters provided by the Japan Meteorological Agency (JMA). It aggregates the daily values into monthly means and returns the resulting time series as an object of class `ts`.

Example

The following example demonstrates how to download SST data for the southern coastal waters of Ibaraki Prefecture, Japan.

The argument `sea_area_id` specifies the numeric identifier of a sea area defined by JMA. For example, 138 corresponds to the coastal waters off southern Ibaraki. A list of available sea area IDs and their corresponding regions is provided by JMA at:

https://www.data.jma.go.jp/kaiyou/data/db/kaikyo/series/engan/eg_areano.html

```
sst_138_ts <- get_jma_sst_ts(sea_area_id = 138)
head(sst_138_ts)
```

```
##           Jan      Feb      Mar      Apr      May      Jun
## 1982 15.04419 14.22500 13.63903 15.31933 17.52258 19.52300
```

4. compute_temp_anomaly()

The `compute_temp_anomaly()` function computes anomalies from a time series provided as an R `ts` object. Anomalies are calculated by subtracting the corresponding long-term mean for each periodic unit, such

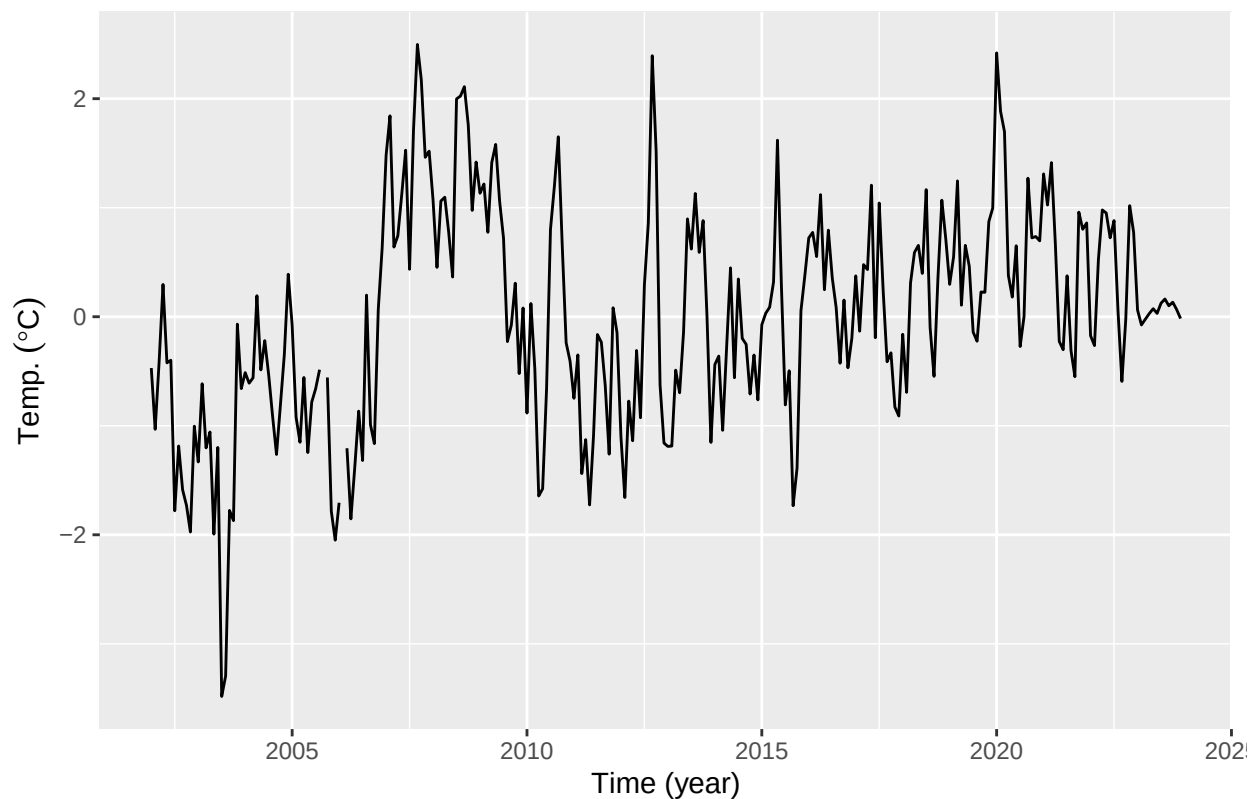
as month, from each observation. This transformation is useful for examining interannual variability after removing periodic effects.

The reference period used to compute anomalies can be specified via a function argument. By default, the climatology is computed over the entire available time series (see `?compute_temp_anomaly`).

This transformation is useful for exploratory analysis and modeling applications that focus on departures from typical seasonal conditions.

Example

```
# Generate temperature anomalies
data(niigata_sst)
niigata_sst_anomaly <- compute_temp_anomaly(niigata_sst)
plt_niigata_sst_anomaly <- forecast::autoplot(niigata_sst_anomaly) +
  labs(y = expression(Temp.~(degree*C)),
       x = "Time (year)")
plot(plt_niigata_sst_anomaly)
```



5. `compute_monthly_climatology()`

The `compute_monthly_climatology()` function computes the climatological mean seasonal cycle from a `ts` object by averaging each seasonal value across years. It is primarily intended for exploratory analysis and visualization of seasonal structure in temperature time series.

Example

```
data(niigata_sst)
monthly_seasonal_cycle_niigata_sst <- compute_monthly_climatology(niigata_sst)
```

```
summary(monthly_seasonal_cycle_niigata_sst)
```

```
##      Month      Temperature
## Min.   : 1.00   Min.   : 9.365
## 1st Qu.: 3.75   1st Qu.:11.169
## Median : 6.50   Median :16.318
## Mean   : 6.50   Mean    :17.036
## 3rd Qu.: 9.25   3rd Qu.:22.294
## Max.   :12.00   Max.    :26.873
```

```
plt_monthly_seasonal_cycle_niigata_sst <- ggplot(
  data = monthly_seasonal_cycle_niigata_sst,
  aes(x = Month, y = Temperature)
) +
  geom_point(size = 2) +
  geom_line(linetype = "dashed") +
  labs(
    title = "Monthly seasonal cycle of temperature",
    x = "Month",
    y = expression(Temp.~(degree*C))
  ) +
  scale_x_continuous(
    breaks = 1:12,
    labels = sprintf("%02d", 1:12)
  ) +
  theme_classic()
```

```
plot(plt_monthly_seasonal_cycle_niigata_sst)
```

